

Actividad Práctica – AEE

Modelado de Comportamiento en Sistemas de E-commerce

1. Introducción y Contexto

En el desarrollo de aplicaciones web y multiplataforma, la lógica de negocio suele implicar tareas que no pueden ocurrir de forma secuencial si queremos una aplicación eficiente. Fíjate que, tras un pago, no podemos hacer esperar al usuario mientras enviamos correos y generamos facturas uno a uno. Aquí es donde el **Diagrama de Actividad de UML 2.5** se convierte en nuestra mejor herramienta de diseño.

2. Objetivos de Aprendizaje

- **Investigar** estándares de modelado en fuentes profesionales y académicas.
- **Diseñar** un flujo de trabajo complejo que gestione decisiones y concurrencia (*Forks* y *Joins*).
- **Documentar** técnicamente el software utilizando el estándar de la industria (GitHub).

3. Desarrollo de la Actividad

(por Fases y en parejas)

Fase 1: Investigación Técnica (40 minutos)

Tarea: Antes de empezar el diseño, debemos documentarnos sobre la sintaxis específica de UML 2.5 para nodos de control. Lo importante aquí es distinguir entre una decisión (rombo) y una bifurcación (barra de *fork*).

Fuentes de consulta obligatorias (Prohibido el uso de IA):

1. **OMG (Object Management Group):** Consultar la especificación formal de UML en <https://www.omg.org/spec/UML/>.
2. **IBM Documentation:** "UML activity diagrams", 2021. Disponible en: <https://www.ibm.com/docs/en/rhapsody/9.0.1?topic=diagrams-uml-activity>.
3. **Visual Paradigm:** "UML Activity Diagram Tutorial". Disponible en: <https://www.visual-paradigm.com/guide/uml/what-is-activity-diagram/>.

Fase 2: Modelado del Proceso de Compra (30 minutos)

Herramientas: Draw.io (app.diagrams.net) o DIA.

Escenario a modelar:

Debéis representar el proceso de "Confirmación de Pedido" de una tienda online:

1. **Entrada:** El usuario pulsa "Finalizar compra".
2. **Validación:** El sistema debe verificar simultáneamente el **Stock** y la **Validez de la Sesión** (Uso de *Fork*).
3. **Pago:** Si ambos son correctos, se procede a la pasarela de pago segura.
4. **Post-Pago (Concurrencia Crítica):** Si el pago es exitoso, el sistema debe ejecutar de forma paralela:
 - Registro del pedido en la base de datos.
 - Generación del PDF de la factura.
 - Envío de notificación por correo electrónico.
5. **Cierre:** Una vez finalizadas *todas* las tareas del punto anterior (Uso de *Join*), se muestra el mensaje de confirmación al cliente.

Fase 3: Entrega Profesional en GitHub (30 minutos)

Un desarrollador profesional se define por cómo entrega su trabajo.

1. Crear un repositorio público en GitHub: EDD_UML_Apellido_Nombre.
2. Estructura del repositorio:
 - /src: Archivo fuente del diagrama (.drawio o .dia).
 - /img: Imagen exportada en alta resolución (PNG o SVG).
 - README.md: Documento principal.

Requisitos del README.md:

- Explicación técnica del proceso modelado.
- Inserción de la imagen del diagrama.
- Justificación del uso de los nodos de sincronización.
- Bibliografía utilizada siguiendo el estilo **IEEE**.



4. Rúbrica de Evaluación

Criterio	Peso	Excelente (9-10)	Adecuado (5-8)	Insuficiente (0-4)
Sintaxis UML	30%	Uso perfecto de nodos iniciales, finales, actividades, <i>forks</i> y <i>joins</i> .	Errores menores en el sentido de las flechas o nombres.	Símbolos incorrectos o mezcla con diagramas de flujo.
Lógica y Concurrency	30%	El paralelismo está correctamente abierto (<i>fork</i>) y cerrado (<i>join</i>).	Hay paralelismo pero falta la sincronización final.	El proceso es lineal o carece de lógica de negocio.
Repositorio GitHub	20%	README impecable con <i>Markdown</i> , imágenes y estructura de carpetas.	README básico con la imagen, sin descripción técnica.	No hay README o el repositorio está vacío.
Investigación (IEEE)	20%	Cita correctamente las fuentes obligatorias y adicionales en formato <i>IEEE</i> .	Cita las fuentes pero no sigue el formato o falta alguna.	No incluye referencias o son enlaces a blogs no profesionales.



5. Notas Importantes

- **Prohibido el uso de IA.** El uso de ChatGPT, Copilot, Gemini o similares para generar el diagrama o el texto del README supondrá el **suspense automático de la actividad**. Queremos evaluar vuestra capacidad de análisis.
- **Entorno.** Se recomienda el uso de **Visual Studio Code** con la extensión de GitHub para gestionar el repositorio.
- **Antes de comenzar.** Levántate, haz que tus compañeros de la clase se enteren de que quieres que te escuchen, y dile al profesor que has comprendido la actividad y el nombre de tu pareja; sin decir nada más, siéntate y comienza con esta actividad (a evaluarse como Competencia transversal).

Relación con el Perfil Profesional: El dominio de UML es una competencia transversal en el desarrollo de software. En entornos laborales reales (Agile, DevOps), estos diagramas sirven de base para la comunicación entre el equipo de diseño y el de backend.

